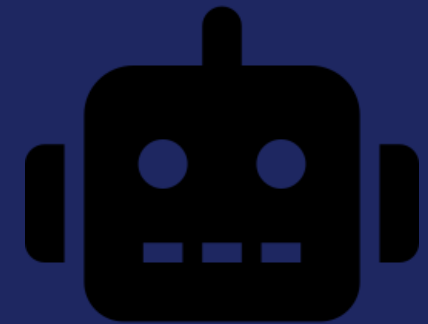


Introduction to Agentic AI



From perception-action loops to LLM-powered agents — the foundational concepts that power autonomous AI systems

8 Concepts

Duration

B2–B3

Bloom's

L2 – Foundational

Level

M1–M4

Prerequisites

Session Overview

What you will learn and why it matters for building production AI systems

What You Will Learn

- What an AI agent is and how it differs from a chatbot
- The perception–decision–action loop that defines all agents
- Reactive agents vs. stateful agents with memory
- How LLMs serve as the reasoning engine inside agents
- SLMs for cost-efficient agent deployment at the edge
- Hands-on: build your first LangChain agent

Concept at a Glance

Concepts: 8 (6 Theory + 1 NEW + 1 Coding)

Bloom's Level: B2–B3 (Understand to Apply)

Difficulty: L2 – Foundational

Module: 5.1 Agent Fundamentals

Prerequisites: Modules 1–4 (GenAI, Architecture, LLMs, Multimodal)

Next: 5.2 Reasoning and Acting (ReAct)

Learning Objectives

By the end of this session you will be able to:

8 Concepts
Duration

B2–B3
Bloom's

L2
Level

M1–M4
Prerequisite

- 1 Define what an AI agent is and explain how it differs from a standard chatbot or LLM wrapper
- 2 Describe the four-component agent anatomy: perception, memory, reasoning, and action
- 3 Contrast reactive agents (no state) with memory-augmented agents (stateful across turns)
- 4 Explain how LLMs function as the reasoning engine inside modern AI agents
- 5 Evaluate when to use SLMs vs. large models for agentic tasks based on cost, latency, and accuracy
- 6 Build a simple ReAct-style agent using LangChain with tool calling and trace logging

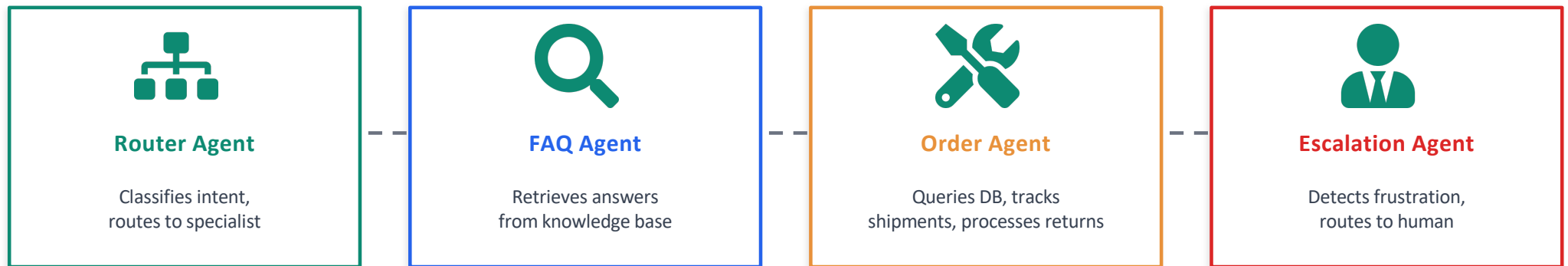
INTRODUCTORY USE CASE

Multi-Agent AI in the Real World

A practical walkthrough before we define the theory

Real-World Use Case: AI-Powered Customer Support

How Klarna replaced 700 human agents with a multi-agent AI system handling 2.3 million conversations/month



Why This Matters — The Numbers

2.3M

conversations
per month

< 2 min

avg resolution
time (was 11 min)

\$40M

annual savings
projected

25%

fewer repeat
inquiries

Anatomy of the Klarna Router Agent

Mapping the real system to the four agent components you will learn today



Perception

Customer message + order history + sentiment signal



Memory

Conversation history (short-term) + CRM profile (long-term)



Reasoning

LLM classifies intent: billing? shipping? complaint? → picks agent

Quick Math: If 2.3M conversations/month, each agent loop averages 4 reasoning steps → 9.2M LLM calls/month. At \$0.01/call (GPT-4o-mini) = \$92K/month. With SLMs for routing at \$0.001/call = \$9.2K/month — a 10× cost reduction. This is why model selection matters.

CONCEPT 1 — WHAT IS AN AI AGENT?

What is an AI Agent?

Defining the building block of autonomous AI systems

What is an AI Agent?

An autonomous system that perceives, reasons, and acts to achieve goals

WHY — The Problem

Standard LLMs are **stateless** — they answer one question and forget. They cannot browse the web, query databases, or take multi-step action. A chatbot answers; an **agent solves**.

WHAT — The Definition

An AI Agent is a software system that uses an LLM as its reasoning engine to: **(1) perceive inputs, (2) maintain state, (3) reason about goals, (4) take actions via tools, and (5) iterate until the task is complete.**

HOW — Chatbot vs. Agent

Dimension	Chatbot	AI Agent
Turns	Single turn	Multi-step loop
Tools	None	Web, DB, APIs, code
Memory	Context window only	Short + long term
Decision	Respond to prompt	Plan → act → observe → repeat
Example	ChatGPT (basic)	Devin, Claude Code, SWE-agent

Agent Anatomy: Perception, Decision, Action

The four-component loop that defines every agent — from simple thermostats to GPT-4 powered systems



Perception

Receives observations: user input, tool outputs, environment state. The agent's 'eyes and ears'.



Memory

Short-term (context window) and long-term (vector DB). Tracks conversation and past tasks.



Reasoning

The LLM interprets observations + memory, decides the next action. This is the 'brain'.



Action

Executes tool calls, writes files, calls APIs, generates output. Then observes the result.

Practical Example — GitHub Copilot Workspace: Perception = issue description + codebase. Memory = file AST index. Reasoning = GPT-4 plans edits. Action = writes code, runs tests. Loop repeats until tests pass.

Simple Reactive Agents

Condition → Action rules with zero internal state — the simplest agent architecture

How It Works

IF user says 'refund' → **THEN** call refund_api()

IF temp > 25°C → **THEN** turn_on(AC)

Limitations

- ✗ No memory — cannot track multi-turn conversations
- ✗ No planning — handles one trigger at a time
- ✗ Brittle — fails on anything outside the rule set
- ✗ No learning — same response every time

Practical Example — Smart Thermostat

Percept: temperature sensor = 28°C

Rule: IF temp > 25 → AC ON

Action: turn_on(AC) ✓

Next percept: temp = 23°C → AC OFF

No memory of previous state is used.

When Reactive Agents Are Enough

- ✓ Simple routing / classification tasks
- ✓ IoT devices with fixed rule sets
- ✓ Guard agents that filter inputs before a smarter agent

Agents with Memory

Adding state transforms a reactive system into one that tracks progress, learns, and adapts

Short-Term (In-Context) Memory

- Everything in the current context window
- Conversation history + tool outputs + system prompt
- Bounded: 4K–2M tokens depending on model
- Lost when session ends
- Fast: zero retrieval latency

Long-Term (External) Memory

- Stored in vector DB (Pinecone, Weaviate) or key-value store
- Retrieved by semantic similarity at each step
- Persistent across sessions — survives restarts
- Scales to millions of entries
- Enables agents to 'remember' users and past tasks

Calculation — Memory Cost Trade-off

Storing 10,000 conversation turns in-context at 100 tokens/turn = 1M tokens → exceeds GPT-4's 128K window. Cost at \$10/1M input tokens = \$10 per query!

Same data in Pinecone: 10K vectors × 1536 dims = 60MB storage at \$0.33/month. Retrieve top-5 = 500 tokens = \$0.005/query. That's a 2,000× cost reduction.

Agent Goals and Objectives

How agents know what to do, when to stop, and whether they succeeded

Explicit Goals

User-provided task:
'Book me a flight to Bangalore
under ₹8,000 for March 15'

Success: flight booked ✓
Fail: no flights under budget ✗

Implicit Goals

Embedded in system prompt:
'You are a helpful customer
support agent for Flipkart'

Success: CSAT > 4.5/5
Fail: escalation triggered

Decomposed Sub-Goals

Agent breaks goal into steps:
1) Search flights 2) Filter price
3) Check dates 4) Book

Each sub-goal has its own
success/fail criteria

⚠️ Goal Drift — The #1 Failure Mode: Over long execution traces (20+ steps), agents gradually shift away from the original intent. Example: asked to 'summarize Q3 earnings', the agent starts researching Q4 predictions after step 12. **Mitigation:** re-inject the original goal every N steps + add a 'goal consistency check' at each reasoning step.

LLMs as Agent Brains

Why large language models make excellent reasoning engines for autonomous agents



Natural Language Understanding

Parse complex, ambiguous task descriptions



Structured Tool Calls

Generate JSON function calls natively via instruction tuning



Chain-of-Thought Reasoning

Plan multi-step solutions before acting



Context Management

Track conversation state across reasoning steps

Key Insight: Agents simulate purposeful behaviour through next-token prediction conditioned on goal descriptions. There is no deep comprehension — but the patterns are rich enough for remarkable multi-step task completion. The LLM is a controller, not a thinker.

Practical: Which LLM for Which Agent Task?

Task	Best Model	Why	Cost/call
Complex planning	GPT-4o / Claude Opus	Best reasoning quality	~\$0.03
Tool routing	GPT-4o-mini / Haiku	Fast, cheap, accurate enough	~\$0.001
Code execution	Claude Sonnet / GPT-4o	Strong code generation	~\$0.01
Simple classification	Llama 3 8B (local)	Free, private, fast	~\$0.0001

SLMs as Lightweight Agent Engines

NEW

The massive industry trend toward tiered model routing — big brain for planning, small brain for execution

WHY — The Cost Problem

A ReAct agent averaging 8 steps/task with GPT-4o at \$0.03/call = \$0.24/task. At 100K tasks/day = \$24,000/day. Replacing 6 of 8 steps with a \$0.001 SLM call drops it to \$0.066/task = \$6,600/day — a 3.6× savings.

Tiered Model Routing Pattern

Tier 1 (Planning): GPT-4o / Claude Opus

Tier 2 (Tool calls): GPT-4o-mini / Haiku

Tier 3 (Classification): Phi-3 / Llama 3 8B (local)

SLM Landscape for Agentic Tasks

Model	Params	VRAM	Latency	Best Agentic Use
Phi-3 Mini	3.8B	~8 GB	~50 ms	On-device classification, intent routing
Llama 3 8B	8B	~16 GB	~80 ms	Tool call formatting, JSON extraction
Mistral 7B	7B	~14 GB	~70 ms	Code generation, structured output
TinyLlama	1.1B	~2 GB	~15 ms	Guard agents, simple binary decisions

Calculation — Latency Impact: GPT-4o p50 latency ≈ 800 ms/call × 8 steps = 6.4 sec/task. With tiered routing (2 GPT-4o + 6 Phi-3): (2×800) + (6×50) = 1,900 ms = 1.9 sec/task. User-perceived latency drops 3.4× while quality stays high for the planning steps that matter most.

Building a Simple Agent

Hands-on: your first LangChain agent with tool calling — 20-minute coding lab

Python — LangChain ReAct Agent

```
from langchain.agents import create_react_agent
from langchain_openai import ChatOpenAI
from langchain.tools import Tool

# 1. Define tools the agent can use
def search(query): ...
def calculate(expr): ...

tools = [
    Tool(name="search", func=search,
         description="Search the web"),
    Tool(name="calc", func=calculate,
         description="Do math"),
]

# 2. Create agent with LLM brain
llm = ChatOpenAI(model="gpt-4o-mini")
agent = create_react_agent(llm, tools)

# 3. Run the agent loop
result = agent.invoke("What is the
population of Tokyo divided by 3?")
```

What Happens Inside

- 1 **Thought:** I need Tokyo's population. Let me search.
- 2 **Action:** search("Tokyo population 2024")
- 3 **Observation:** Tokyo: 13.96 million (2024)
- 4 **Thought:** Now I divide by 3.
- 5 **Action:** calc("13960000 / 3")
- 6 **Observation:** 4,653,333.33
- 7 **Answer:** $\approx$ 4.65 million

Key Takeaways

The six essential points from Sub-Module 5.1

Agents ≠ Chatbots

An agent maintains a goal-directed loop: perceive → reason → act → observe → repeat. A chatbot responds once.

Four components define every agent

Perception, Memory, Reasoning (LLM), and Action (tools). These compose to handle arbitrarily complex tasks.

Memory transforms capability

Short-term (context) for session state + long-term (vector DB) for persistent knowledge. 2,000× cheaper than stuffing everything in context.

LLMs are controllers, not thinkers

They simulate purposeful behaviour via next-token prediction. Remarkable capability, but no deep comprehension.

SLMs slash cost and latency

Tiered routing (big model plans, small model executes) reduces cost 3–10× and latency 3× with minimal quality loss.

Next: ReAct framework (5.2)

The formal Thought-Action-Observation loop that structures everything you learned today into a production pattern.

AI-AG-FN-TH-000001 → CD-000001 Complete

Introduction to Agentic AI

Sub-Module 5.1: Agent Fundamentals

Advanced Certification in Agentic & Generative AI · IISc Bangalore

Prof. Sashikumaar Ganesan · AhamX Bodhi